Evolução da Qualidade Estrutural de Projetos Spring: transição para Spring Boot e evidências de assistência por IA

Gabriel Ferreira[1], **Guilherme Augusto**[1]

[1]Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC-MG)
Belo Horizonte – MG – Brazil

{gabriel.amaral.1447804,gajsouza}@sga.pucminas.br

Abstract. *This observational longitudinal study investigates the structural evolution of Java repositories in the Spring ecosystem created through 2017 and active during Era E4 (`pushed_at >= 2022-01-01`). Rather than filtering by star counts, repository relevance was assessed locally across 785 candidates using four complementary RepoReaper curation scenarios (non-didactic purpose, non-toy projects adjusted to preserve Spring Boot `demo`/`sample`/`example` naming conventions, standardized Maven/Gradle build descriptors, and automated test presence), approving 607 repositories. Requiring complete and valid CK metric extraction across every era of each repository's lifecycle yielded 470 valid longitudinal repositories (1,427 CK snapshots): 17 in the pioneer group (2009–2013) tracked continuously across four eras (E1–E4) and 453 in the main group (2014–2017) tracked continuously across three eras (E2–E4). Longitudinal comparisons across the exact same repositories revealed monotonic growth in coupling (CBO) and class response (RFC), accompanied by slight reductions in class complexity (WMC) and lack of cohesion (LCOM*) and stable inheritance hierarchies. In E4, valid repositories with detected AI traces ($n = 75$) exhibited higher metric averages than those without detected traces ($n = 395$), alongside substantially larger codebases. Findings describe historical trajectories and observed associations without claiming causal effects of Spring Boot or AI.*

Resumo. *Este estudo observacional longitudinal investiga a evolução estrutural de repositórios Java do ecossistema Spring criados até 2017 e ativos na era E4 (`pushed_at >= 2022-01-01`). Em vez de aplicar cortes por número de estrelas, a relevância de engenharia dos 785 candidatos foi aferida localmente pelos quatro cenários conjuntos do RepoReaper (finalidade não didática, exclusão de projetos brinquedo flexibilizada para preservar as convenções `demo`/`sample`/`example` do Spring Boot, automação de compilação Maven/-Gradle e presença de testes automatizados), aprovando 607 repositórios. A exigência metodológica de extração válida do CK em todas as eras previstas para cada repositório definiu a amostra final de 470 repositórios válidos (1.427 snapshots extraídos com CK): 17 no grupo pioneiro (2009–2013) acompanhados continuamente nas quatro eras (E1–E4) e 453 no grupo principal (2014–2017) acompanhados continuamente nas três eras (E2–E4). As comparações longitudinais sobre os mesmos projetos indicaram crescimento consistente de acoplamento (CBO) e resposta por classe (RFC), coexistindo com leve redução*

de complexidade ponderada (WMC) e falta de coesão (LCOM) e estabilidade em herança. Em E4, sistemas válidos com evidência detectada de IA ($n = 75$) exibiram médias superiores nas sete métricas frente aos sistemas sem evidência detectada ($n = 395$), mas também bases de código expressivamente maiores. Os resultados descrevem trajetórias e associações observadas, sem atribuir efeitos causais isolados ao Spring Boot ou à IA.*

Bacharelado em Engenharia de *Software* - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): Danilo Maia - danilomaia@pucminas.br
Orientador de conteúdo (TCC I): João Pedro Batisteli - joaobatisteli@pucminas.br
Orientador de conteúdo (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br
Orientador acadêmico (TCC I): Cleiton Tavares - cleitontavares@pucminas.br
Belo Horizonte, versão de trabalho de 29 de outubro de 2026.

1. Introdução

A qualidade interna de *software* descreve propriedades estruturais do código-fonte, como modularidade, coesão e acoplamento, que podem ser investigadas por análise estática [ISO/IEC 2011, Pressman and Maxim 2020]. Em sistemas orientados a objetos, a suíte de Chidamber e Kemerer oferece indicadores para caracterizar essas dimensões [Chidamber and Kemerer 1994]. Esses indicadores não constituem, isoladamente, uma nota de maturidade: sua interpretação depende do tamanho, do domínio e da organização do projeto.

O ecossistema Spring permite investigar a evolução de projetos Java ao longo de diferentes períodos tecnológicos. Projetos iniciados antes da consolidação do Spring Boot podem ter atravessado mudanças de configuração, dependências, práticas de desenvolvimento e ferramentas de assistência. O Spring Boot 1.0 estável foi lançado em 1ž de abril de 2014 [Spring 2014]; por isso, o período de 2009 a 2013 é tratado como contexto de Spring tradicional, com identificação explícita de eventuais versões preliminares de Boot. A presença atual de Boot em um repositório criado em 2010 não demonstra que essa tecnologia já integrava seu código em 2010.

A mesma distinção é necessária para a inteligência artificial. Um projeto antigo pode apresentar hoje vestígios de ferramentas de IA introduzidos recentemente. Datas de criação dos repositórios não datam o uso dessas ferramentas, e fronteiras de calendário não separam perfeitamente produção humana e assistida. O Copilot já estava em prévia técnica em junho de 2021 [GitHub 2021]. Evidências de arquivos, identidades e mensagens de commit permitem observar parte dessa adoção, mas não todo uso nem a proporção do código produzida por IA [Robbes et al. 2026].

A pergunta central deste trabalho é: **como a qualidade estrutural de projetos do ecossistema Spring evoluiu ao longo do tempo, e quais mudanças acompanharam a adoção do Spring Boot e o aparecimento de evidências de assistência por IA?** O foco recai no acompanhamento longitudinal dos mesmos projetos criados até 2017 e ativos na era contemporânea, aferindo sua relevância de engenharia por curadoria multidimensional baseada no RepoReaper em vez de limiares de estrelas. Os projetos pioneiros criados

até 2013 permitem observar versões anteriores à consolidação do Boot, ao passo que os iniciados entre 2014 e 2017 caracterizam a evolução a partir de seu lançamento. Esse desenho distingue evolução interna do mesmo repositório de diferenças de composição entre amostras transversais.

O objetivo geral é caracterizar longitudinalmente a evolução estrutural dos projetos selecionados, recuperando a última versão disponível dentro de cada era do histórico Git (em E4, o HEAD da branch principal na coleta). Os objetivos específicos são: (i) descrever trajetórias de acoplamento, complexidade, tamanho, herança e coesão; (ii) descrever mudanças nos mesmos projetos entre versões com diferentes configurações de Spring e Boot; (iii) explorar as mudanças que acompanham o aparecimento de evidências de IA; e (iv) situar essas trajetórias no contexto dos projetos disponíveis em cada época, por tamanho e framework. O paradigma GQM liga essas perguntas às sete medidas extraídas pela ferramenta CK. O acompanhamento histórico constitui o eixo principal; Boot e evidências de IA são características observáveis cuja análise depende da cobertura disponível. Os pontos de encerramento das eras não identificam necessariamente o instante exato de uma transição.

O interesse na evolução da comunidade é abordado por esse recorte de projetos, sem tratar a amostra como representação de todos os usuários de Spring. A idade de um repositório, a adoção de uma ferramenta e a redução de uma métrica não são equivalentes a maturidade. Neste trabalho, qualidade estrutural designa as dimensões do código Java observadas pelos indicadores; não inclui uma avaliação direta de produtividade, funcionalidade, desempenho ou competência dos desenvolvedores.

A relevância do estudo está em produzir evidências históricas rastreáveis, com código, datas, versões e cobertura associados a cada medição. A mineração de repositórios possibilita esse acompanhamento, mas exige cuidado com seleção, disponibilidade do histórico e interpretação dos metadados [Hassan 2008, Kalliamvakou et al. 2016]. Mudanças de framework, linguagem, domínio, equipe e tamanho são explicações concorrentes para variações das métricas. Nem a injeção de dependências elimina a influência do framework sobre a estrutura, nem um contraste entre períodos permite atribuir diferenças exclusivamente ao desenvolvedor ou à IA.

As saídas incluem tabelas de valores médios por era e de diferenças longitudinais sobre os mesmos projetos, com registros de reprodução. Este artigo apresenta a análise longitudinal dos 470 repositórios válidos finais que superaram a curadoria em quatro cenários a partir dos 785 candidatos históricos e mantiveram dados estruturais completos do CK em todas as eras de seu respectivo grupo temporal. A investigação admite estabilidade, aumentos e reduções das métricas, sem presumir melhoria. A Seção 2 apresenta a fundamentação; a Seção 3, os trabalhos relacionados; a Seção 4, o protocolo histórico; a Seção 5, os resultados obtidos; e a Seção 6, as conclusões e seus limites.

2. Fundamentação Teórica

Esta seção apresenta os conceitos que embasam o estudo: qualidade estrutural de *software*, métricas orientadas a objetos, o paradigma *Goal-Question-Metric*, o acompanhamento histórico por mineração de repositórios e as evidências de assistência por inteligência artificial.

2.1. Qualidade interna e métricas orientadas a objetos

A norma *ISO/IEC 25010* define qualidade de *software* como um conjunto de características que expressam o grau em que um sistema atende a necessidades explícitas e implícitas [ISO/IEC 2011]. Nesse modelo, a qualidade interna refere-se a propriedades do código-fonte observáveis por análise estática, incluindo modularidade, coesão e facilidade de manutenção. Pressman e Maxim destacam que sistemas com melhor qualidade interna tendem a exigir menor esforço de evolução e a permanecer viáveis por mais tempo [Pressman and Maxim 2020].

Para tornar essas características mensuráveis em código orientado a objetos, Chidamber e Kemerer propuseram um conjunto clássico de indicadores em nível de classe [Chidamber and Kemerer 1994]. Acoplamento entre objetos (*Coupling Between Objects* — CBO) descreve o grau de dependência de uma classe em relação a outras. Métodos ponderados por classe (*Weighted Methods per Class* — WMC) agrega a complexidade dos métodos. Profundidade da árvore de herança (*Depth of Inheritance Tree* — DIT) e número de filhos (*Number of Children* — NOC) caracterizam a hierarquia. Resposta de uma classe (*Response for a Class* — RFC) estima o conjunto de métodos potencialmente acionados a partir de uma mensagem. Falta de coesão em métodos (*Lack of Cohesion in Methods* — LCOM) indica o quanto os métodos de uma classe operam sobre conjuntos disjuntos de atributos. Essas métricas passaram a ser amplamente empregadas para caracterizar projeto, manutenção e falhas em sistemas orientados a objetos [Singh et al. 2010, Radjenović et al. 2013, Aniche et al. 2016]. A forma operacional em que são extraídas neste trabalho, incluindo a ferramenta e as variantes adotadas, é especificada na Seção 4.

Neste estudo, WMC representa a complexidade estrutural agregada dos métodos, conforme a implementação do CK; não mede tempo de execução, consumo de recursos ou desempenho computacional. O CBO empregado é o indicador geral de dependências entre classes do CK, sem uma separação implementada entre acoplamento horizontal e vertical. Para coesão, utiliza-se a coluna `lcom*`, variante normalizada documentada pelo instrumento, e não a coluna `lcom` original. As seis medidas em nível de classe são complementadas por LOC por método; essa medida de tamanho não é apresentada como uma sétima integrante da suíte CK original.

Valores devem ser interpretados em conjunto e segundo a função do código. Uma redução de DIT ou NOC descreve uma mudança na hierarquia, sem estabelecer que o projeto se tornou melhor. Mesmo mudanças em complexidade ou acoplamento exigem contexto sobre responsabilidades e tamanho [Aniche et al. 2016]. Nenhuma métrica isolada prova violação de um princípio de projeto, nem identifica a autoria humana ou assistida do código.

2.2. Goal-Question-Metric

O paradigma *Goal-Question-Metric* (GQM) organiza avaliações de *software* em três níveis: o objetivo do estudo (*goal*), as perguntas que o detalham (*questions*) e as métricas que as respondem (*metrics*) [Basili et al. 1994]. A abordagem evita coletar indicadores sem vínculo com a intenção da pesquisa: primeiro define-se o que se deseja aprender; em seguida, traduz-se esse objetivo em perguntas mensuráveis; por fim, escolhem-se as medidas adequadas. Neste trabalho, o GQM liga o objetivo de analisar a evolução da

qualidade estrutural em projetos do ecossistema Spring às dimensões de acoplamento, complexidade, herança, coesão e tamanho, operacionalizadas na Seção 4.

2.3. Mineração de Repositórios de Software

A Mineração de Repositórios de *Software* (*Mining Software Repositories* — MSR) analisa artefatos reais — código-fonte, histórico de versões e metadados de plataformas — para identificar padrões de desenvolvimento, evolução e qualidade [Hassan 2008]. Em vez de apoiar-se apenas em relatos ou documentação, a MSR observa evidências extraídas dos próprios repositórios. No GitHub, essa estratégia exige exclusão de *forks*, distinção entre criação na plataforma, idade do código e disponibilidade de versões, bem como critérios de relevância ancorados em práticas de engenharia em vez de indicadores de popularidade social (como contagem de estrelas), evitando excluir sistemas reais de menor visibilidade ou interpretar metadados atuais como evidência histórica [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016, Munaiah et al. 2017]. Esses cuidados conceituais orientam a amostragem e a curadoria descritas na metodologia.

2.4. Inteligência artificial generativa

Neste trabalho, inteligência artificial generativa designa sistemas baseados em modelos de linguagem de grande escala, treinados em corpora de texto e código, capazes de produzir ou completar trechos de programa a partir de contexto e de *prompts*. Ferramentas como GitHub Copilot, ChatGPT, Claude Code e Cursor ilustram formas distintas de assistência, introduzidas em momentos diferentes. E4, iniciada em 2022, é um recorte de calendário do estudo; pertencer a essa época não confirma uso de IA, assim como estar em E3 não o exclui.

Experimentos sobre produtividade e qualidade de sugestões motivam investigar essas ferramentas, mas medem resultados diferentes das métricas estruturais de um repositório [Peng et al. 2023, Moradi Dakhel et al. 2023]. Também é necessário distinguir assistentes de completamento e agentes de codificação: estes podem deixar rastros mais explícitos, como coautoria em commits [Robbes et al. 2026]. Consequentemente, a distribuição temporal dos vestígios encontrados depende tanto da adoção quanto da possibilidade de detectá-la. O estudo acompanha evidências observáveis, sem equipará-las a todo uso de IA.

3. Trabalhos Relacionados

Os trabalhos relacionados são organizados em quatro frentes: acompanhamento histórico de sistemas, curadoria de repositórios de software, interpretação de métricas estruturais e evidências de assistência por IA. Essa organização situa a contribuição principal do TCC na evolução de projetos Spring, tratando a IA como uma possível mudança posterior em suas trajetórias.

3.1. Evolução de sistemas e mineração de repositórios

[Godfrey and Tu 2000] acompanharam 96 versões do núcleo Linux, investigando o crescimento do sistema e de seus subsistemas. O estudo encontrou trajetórias que exigiam examinar a organização interna, além dos valores agregados de tamanho. A relação com este TCC é metodológica: observar versões do mesmo sistema permite investigar mudanças que uma única medição atual não revela. O objeto e as medidas diferem, pois o

presente trabalho acompanha vários projetos Java/Spring por snapshots ao final de cada era e métricas estruturais, sem transferir o padrão de crescimento do Linux para esse ecossistema.

A mineração de repositórios oferece meios para reconstruir essas trajetórias, mas a disponibilidade de dados não garante sua adequação à pergunta de pesquisa [Hassan 2008]. Os estudos de [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016] discutem problemas de interpretação dos repositórios do GitHub e de sua relação com a atividade de desenvolvimento. Esse cuidado fundamenta a distinção, neste trabalho, entre um candidato encontrado por metadados, um histórico recuperável e uma versão tecnicamente elegível. A seleção original já excluiu arquivados e projetos sem atividade recente pelos metadados. Manter essa base assegura rastreabilidade da população autorizada, mas preserva o viés de sobrevivência e não recupera projetos desaparecidos.

3.2. Curadoria de repositórios de software

[Munaiah et al. 2017] propõem um arcabouço para identificar projetos de software de engenharia em meio aos repositórios do GitHub e implementam a ferramenta `reaper`, que avalia sinais objetivos de práticas do projeto. Os autores demonstram que filtros baseados em número de estrelas constituem aproximações inadequadas de relevância de engenharia: limiares de popularidade descartam ampla parcela de sistemas reais mantidos por equipes independentes ou organizações com baixa exposição social, ao passo que retêm repositórios populares de cunho didático ou documental, preocupação também apontada por [Kalliamvakou et al. 2016]. A relação com este TCC é direta: adota-se a curadoria inspirada no RepoReaper — e não um piso de estrelas — como critério de relevância dos repositórios, combinando quatro cenários complementares (finalidade não didática, exclusão de projetos brinquedo/efêmeros, automação de compilação e presença de testes automatizados). Ademais, a operacionalização local flexibiliza vocábulos que no ecossistema Spring Boot designam arquiteturas reais ou nomes padrão de inicialização (`demo`, `sample` e `example`). Os parâmetros e essa adaptação lexical constituem decisões operacionais deste TCC; os resultados de [Munaiah et al. 2017] sustentam o conceito de curadoria multidimensional, sem validar especificamente esses ajustes para projetos Spring nem substituir revisão humana em casos limítrofes.

3.3. Interpretação contextual das métricas

A abordagem SATT, de [Aniche et al. 2016], propõe limites de métricas específicos para papéis arquiteturais, em vez de avaliar todas as classes pelos mesmos valores de referência. O estudo examinou projetos MVC e Android e relacionou diferenças de métricas às responsabilidades das classes. Para este TCC, a implicação é que uma variação agregada pode refletir mudanças na composição arquitetural do projeto. O trabalho não aplica os limites SATT nem classifica automaticamente os projetos como bons ou ruins: utiliza as mesmas medidas ao longo do tempo e explicita o contexto de sua interpretação.

No contexto de injeção de dependências, [Sun et al. 2022] propõem medidas que distinguem formas de acoplamento relevantes à manutenibilidade. A investigação mostra por que métricas de dependência precisam ser interpretadas à luz dos mecanismos utilizados no código. Este TCC mantém o CBO geral do instrumento CK; portanto, não reproduz as medidas ponderadas daquele estudo nem assume que a injeção de dependências elimina o acoplamento observado. A comparação de variantes de CBO em

[Child et al. 2019] também reforça a necessidade de documentar qual definição foi operacionalizada.

Esses trabalhos delimitam o alcance das conclusões: métricas estruturais descrevem propriedades do código, enquanto sua relação com qualidade depende do contexto. A consistência do extrator entre versões ajuda a comparar medições, mas não transforma uma redução numérica em evidência suficiente de melhoria da arquitetura.

3.4. Assistência por IA e identificação de vestígios

A evolução recente dos Modelos de Linguagem de Grande Porte (LLMs) aplicados à engenharia de software transformou as práticas de programação a partir de 2022 [Hou et al. 2024]. Estudos empíricos e experimentais sobre o GitHub Copilot e assistentes baseados em código analisam ganhos de produtividade, custos cognitivos na interação com agentes e correção funcional em tarefas delimitadas [Peng et al. 2023, Moradi Dakhel et al. 2023, Yetistiren et al. 2023, Mozannar et al. 2024]. O EvalPlus, apresentado por [Liu et al. 2023], amplia os testes usados para avaliar a correção funcional de soluções geradas. Esses resultados motivam a investigação da assistência por IA, mas não fornecem uma previsão direta para as métricas CK de sistemas completos de grande porte. O presente estudo observa repositórios históricos reais; não reproduz os experimentos controlados nem mede tempo de desenvolvimento ou correção funcional.

A abordagem BIMAN, de [Dey et al. 2020], combina características de nomes, mensagens e atividade para identificar bots que realizam commits. Essa identificação não equivale à detecção de assistência por IA: automações convencionais também produzem commits, e contribuições assistidas podem permanecer sob a identidade do desenvolvedor. Por isso, um autor automatizado precisa de interpretação contextual e a ausência de uma identidade de bot não caracteriza um projeto como exclusivamente humano.

[Robbes et al. 2026] investigam vestígios de agentes de codificação em 128.018 projetos do GitHub, distinguindo-os de assistentes de completamento. O estudo explora rastros como configuração e coautoria, que tornam parte dessa adoção observável. O detector deste TCC é uma adaptação local, com heurísticas próprias e inspeção temporal do histórico Git; sua precisão não é herdada daquele trabalho. Sinais de configuração, identidades e trailers são conservados como evidências distintas, sem determinar a parcela de código gerada ou a continuidade do uso.

A contribuição deste trabalho está na combinação entre versões históricas de projetos Spring, medições estruturais rastreáveis e marcos observáveis de framework e assistência por IA. A evolução dos projetos constitui o objeto principal. Os contrastes em torno de Boot e IA complementam essa descrição, sujeitos à quantidade e à qualidade das versões e evidências disponíveis.

4. Materiais e Métodos

Este estudo adota delineamento observacional, longitudinal e descritivo sobre repositórios históricos do ecossistema Spring no GitHub. A unidade acompanhada é o repositório, e a unidade medida é o último snapshot disponível na linha principal dentro de cada era (em E4, o commit HEAD da branch principal coletado em outubro de 2026). As comparações caracterizam trajetórias estruturais ao longo do tempo e associações observadas;

não estabelecem efeitos causais isolados do Spring Boot ou de ferramentas de inteligência artificial.

4.1. Base de origem, relevância sem filtro de estrelas e recorte de candidatos

A população de partida provém dos 95.894 registros únicos consolidados em `population.csv` (execução `20260920_java`), recuperados via API REST do GitHub pela consulta `topic:spring-boot fork:true`, particionada por intervalos de data de criação e, apenas quando exigido pelo limite técnico de mil itens por janela de busca da API, subdividida por faixas de estrelas sem descarte de elegíveis. O tópico identifica candidatos potenciais e não substitui a verificação estrutural do repositório.

Sobre os 95.894 registros únicos aplicaram-se filtros sequenciais e determinísticos de metadados, contabilizando cada repositório na primeira condição de exclusão: linguagem principal ausente ou distinta de Java (23.298), *forks* (1.089), espelhos (zero), arquivados (1.607), desabilitados (zero), atividade ausente ou anterior à abertura da era E4 em `2022-01-01T00:00:00Z` (16.615), *templates* (323), repositórios vazios ou sem branch padrão (três) e materiais retidos pelas heurísticas automáticas iniciais de tópicos e prefixos estritamente escolares (540). As 43.475 exclusões iniciais resultam em 52.419 repositórios Java ativos durante a era E4. Em seguida, aplica-se o corte histórico de criação até o encerramento de 2017 em UTC (`created_at < 2018-01-01T00:00:00Z`), excluindo 51.634 repositórios mais recentes e definindo a base longitudinal de **785 repositórios candidatos**: 22 criados até 2013 e 763 criados entre 2014 e 2017.

Diferentemente de levantamentos de mineração de repositórios que impõem pisos arbitrários de popularidade (como o corte usual de dez ou mais estrelas), **este trabalho não aplica filtro algum por número de estrelas**. Contadores de popularidade no GitHub refletem exposição social, divulgação comunitária ou uso didático viral, mas não aferem práticas de engenharia de software [Kalliamvakou et al. 2016, Munaiah et al. 2017]. Na população de 785 candidatos, 389 repositórios (49,6%) possuem menos de dez estrelas (incluindo 156 com zero estrelas e 73 com uma estrela) e 396 (50,4%) possuem dez ou mais estrelas. Descartar projetos de baixa contagem de estrelas eliminaria metade do ecossistema real — sobretudo microsserviços corporativos, integrações internas e sistemas mantidos por desenvolvedores independentes. Em substituição ao critério de estrelas, **a relevância de cada repositório é determinada pela curadoria multidimensional de práticas de engenharia baseada no RepoReaper**, detalhada na Seção 4.3.

4.2. Janela de atividade na era E4 e fundamentação temporal

O requisito de atividade contemporânea adota `pushed_at >= 2022-01-01T00:00:00Z`, alinhando o filtro de sobrevivência operacional exatamente ao início da era E4 (2022–2026). Esse quadriênio corresponde ao marco histórico de disseminação de Modelos de Linguagem de Grande Porte (LLMs), assistentes de completamento de código e agentes autônomos na engenharia de software [Hou et al. 2024, Mozannar et al. 2024, Robbes et al. 2026]. Exigir atividade a partir de janeiro de 2022 — em vez de restringir a busca ao último triênio contado de setembro de 2023 (que reteria apenas 540 candidatos) ou a janelas curtas de um ano ou menos — resgata 245 sistemas Java longevos criados até 2017 que estiveram sob manutenção efetiva durante a era da assistência generativa.

Essa janela temporal viabiliza a aplicação simultânea de todos os quatro cenários de curadoria do RepoReaper sobre uma amostra histórica robusta, em consonância com precedentes de janelas plurianuais em mineração de projetos Java [Oliveira et al. 2021]. Registra-se, como limitação inerente aos metadados da plataforma, que `pushed_at` funciona como aproximação de atividade (podendo refletir envio de *tags* ou atualização de ramos secundários, e não necessariamente commits contínuos na branch principal) e que a exclusão de repositórios inativos antes de 2022 ou arquivados mantém viés de sobrevivência sobre os sistemas acompanhados.

4.3. Higienização pelos quatro cenários do RepoReaper e flexibilização para Spring Boot

Para assegurar que os candidatos representem projetos de software de engenharia (*Engineered Software Projects*) em vez de exercícios escolares, repositórios efêmeros ou depósitos não estruturados [Kalliamvakou et al. 2016, Munaiah et al. 2017], aplica-se a etapa de higienização baseada no arcabouço RepoReaper. A ferramenta foi operacionalizada **integralmente em ambiente local**, mediante instrumento determinístico e auditável em Python (`reporeaper_curation.py`), sem execução em nuvem ou dependência de serviços externos proprietários.

Em lugar de utilizar contagem de estrelas como proxy de relevância, a elegibilidade na curadoria exige aprovação conjunta nos **quatro cenários complementares (C1 + C2 Ajustado + C3 + C4)**:

1. **Cenário C1 (Curadoria de Finalidade Didática Estrita):** Exclusão sistemática de repositórios voltados a ensino, disciplinas acadêmicas ou treinamento, identificados pelos termos `course`, `homework`, `assignment`, `learning`, `tutorial`, `aula`, `curso`, `exercicio`, `treinamento`, `udemy`, `mooc`, `exercism` e `lecture` em nomes, descrições e tópicos. Expressões técnicas de domínio, como `machine-learning` e `deep-learning`, são preservadas para não descartar aplicações reais de aprendizado de máquina.
2. **Cenário C2 Ajustado (Curadoria de Projetos Brinquedo e Efêmeros, com flexibilização para Spring Boot):** Exclusão de áreas de experimentação descartável, rascunhos de prática e esqueletos sem propósito produtivo, identificados por `playground`, `sandbox`, `toy`, `practice`, `boilerplate`, `cheatsheet`, `cheat-sheet` e `slides`, **excluindo-se da penalização os parâmetros `demo`, `sample` e `example`** (e suas formas plurais).
3. **Cenário C3 (Arquitetura de Build Padronizada):** Exigência de infraestrutura formal de automação de compilação e gerenciamento de dependências do ecossistema Java/Spring, comprovada pela presença de descritores canônicos (`pom.xml`, `build.gradle` ou `build.gradle.kts`).
4. **Cenário C4 (Engenharia de Testes Automatizados):** Exigência de evidência estrutural de verificação automatizada no repositório, aferida pela presença da hierarquia padrão de testes (`src/test`) ou de classes de teste automatizado (`*Test.java`), operacionalizando a dimensão de testes do RepoReaper [Munaiah et al. 2017].

A flexibilização específica do **Cenário C2** para retirar `demo`, `sample` e `example` da lista de bloqueio decorre de particularidades técnicas diretas do ecossis-

tema Spring Boot. Em primeiro lugar, o gerador oficial da comunidade, *Spring Initializr* (`start.spring.io`), atribui por padrão o identificador de artefato e nome inicial `demo` aos projetos gerados; diversos microsserviços e APIs reais evoluem por anos mantendo esse radical no identificador ou na descrição original do repositório. Em segundo lugar, é convenção arquitetural recorrente entre organizações e mantenedores Java nomear implementações completas de microsserviços de referência, servidores de autenticação OAuth2, pipelines de mensageria Kafka e módulos corporativos reutilizáveis com os termos `sample` ou `example`. A verificação empírica sobre a base confirmou essa especificidade: penalizar esses três vocábulos descartaria indevidamente 149 projetos aprovados na curadoria (114 deles com histórico estrutural completo em todas as eras), que possuem automação de build Maven/Gradle, suíte de testes automatizados e manutenção plurianual. A flexibilização elimina esses falsos negativos específicos do domínio Spring Boot sem abrir mão da exclusão rigorosa de sandboxes e repositórios brinquedo.

Nos 785 candidatos avaliados, as aprovações individuais foram de 765 em C1 (97,5%), 768 em C2 Ajustado (97,8%), 752 em C3 (95,8%) e 665 em C4 (84,7%); a interseção estrita dos **quatro cenários juntos aprovou 607 repositórios (77,3%)**: 20 criados até 2013 e 587 criados entre 2014 e 2017.

4.4. Grupos temporais, rastreabilidade de SHAs e critério de validade longitudinal

As quatro eras tecnológicas analisadas são E1 (2009–2013), E2 (2014–2017), E3 (2018–2021) e E4 (2022 até o HEAD da branch principal coletado em outubro de 2026). A data de criação define o grupo temporal de ingresso do repositório, ao passo que o carimbo temporal do commit define a versão efetivamente medida em cada era, evitando tomar código atual como evidência do passado. Em E1, E2 e E3, percorrendo a linha do primeiro pai dos commits de *merge*, seleciona-se o último commit cuja data de committer pertença ao intervalo da era (sem projeção retroativa do HEAD futuro quando não há commit no período); em E4, fixa-se o SHA do HEAD coletado na branch principal.

Para assegurar comparabilidade longitudinal estrita e eliminar flutuações de composição amostral entre os períodos comparados, **um repositório só é considerado válido na amostra analítica final se possuir snapshot recuperável e extração válida de métricas CK em todas as eras previstas para o seu respectivo grupo temporal**. Dos 607 repositórios aprovados no RepoReaper, 137 não atenderam a esse critério de completude longitudinal (122 por ausência de commit registrado na linha principal em alguma era intermediária e 15 por falha de clonagem ou ausência de classes Java mensuráveis em alguma das eras), sendo excluídos da síntese comparativa sem substituição por outros identificadores.

Aplicado esse filtro final de completude longitudinal, a amostra válida do estudo é constituída por **470 repositórios válidos (1.427 snapshots CK completos)**, divididos em dois grupos sem sobreposição:

1. **Grupo Pioneiro (criados até 31/12/2013):** Compreende **17 repositórios válidos** (de 20 aprovados na curadoria dessa faixa), originados antes do lançamento estável do Spring Boot 1.0 e acompanhados com medições completas do CK nas **quatro eras (E_1, E_2, E_3, E_4)**, totalizando 68 snapshots válidos.
2. **Grupo Principal (criados entre 2014 e 2017):** Compreende **453 repositórios válidos** (de 587 aprovados na curadoria dessa faixa), iniciados a partir do surgi-

mento do Spring Boot e acompanhados com medições completas do CK nas **três eras de seu ciclo de vida** (E_2, E_3, E_4), totalizando 1.359 snapshots válidos.

Esse desenho garante que todas as comparações entre eras dentro de cada grupo — E_1 contra E_2, E_3 e E_4 no grupo pioneiro ($n = 17$), bem como E_2 com E_3, E_3 com E_4 e E_2 com E_4 no grupo principal ($n = 453$) — operem exatamente sobre a mesma quantidade e os mesmos identificadores de repositórios.

4.5. Extração estática CK e medidas contextuais de porte

As métricas estruturais são extraídas estaticamente sobre cada árvore de código recuperada via `checkout` determinístico do SHA, utilizando o CK 0.7.1-SNAPSHOT (revisão `8a1ef916`), apoiado no Eclipse JDT 3.44.0 (configuração Java 21) sob JVM OpenJDK 17 [Aniche 2015], sem compilação ou execução dos sistemas. Preservam-se as médias por classe de CBO geral, RFC, WMC, DIT, NOC e LCOM*, acrescidas de LOC por método, mantendo dados brutos e contagens de produção e testes discriminados.

Operacionalmente, CBO contabiliza acoplamento de tipos resolvidos pela AST; RFC conta invocações distintas estaticamente identificáveis; WMC soma ramificações de complexidade estrutural por classe; DIT parte de base unitária na raiz; NOC mede subclasses diretas no snapshot; e LCOM* emprega a formulação normalizada $\frac{1}{a}\sum_{i=1}^{a}\frac{m-\mu(A_i)}{m}$, em que a representa atributos, m métodos e $\mu(A_i)$ métodos que acessam cada atributo. Classes em que LCOM* é matematicamente indefinido são omitidas apenas do denominador dessa métrica, sem imputação por zero; igualmente, falhas de extração nunca são convertidas em métrica zero.

Não se adota corte mínimo de porte (seja por volume de linhas de código ou por quantidade de arquivos) para inclusão dos repositórios: a validação de relevância concentra-se nos quatro cenários do RepoReaper aliados à completude longitudinal nas eras, permitindo observar fases iniciais genuínas e microsserviços enxutos sem viés de truncamento. O tamanho físico de cada versão é registrado como variável contextual descritiva, contando cada caminho distinto de arquivo `.java` uma única vez e somando linhas físicas não vazias e não exclusivamente comentadas sem duplicação por classes internas, em alinhamento com as recomendações de definição de contagem de Park [Park 1992].

4.6. Evidências de IA, comparações longitudinais e reprodutibilidade

A detecção local de assistência por IA inspeciona artefatos de configuração de agentes e assistentes (como diretórios e instruções dedicadas de GitHub Copilot, Cursor, Claude Code e Codex/OpenAI), regras em `.gitignore`, identidades de bots de codificação e *trailers* `Co-authored-by` no histórico Git disponível até o snapshot avaliado [Robbes et al. 2026]. A ausência de vestígios explícitos não comprova ausência de uso de IA, tampouco o primeiro rastro versionado data o início exato da adoção.

A síntese estatística é estritamente descritiva, atribuindo peso igual a cada repositório válido nas médias por era e omitindo p-valores ou medidas inferenciais como delta de Cliff. Como cada grupo reúne exclusivamente repositórios com medições CK válidas em todas as suas eras, os resultados são apresentados em pares complementares de tabelas: uma com os valores médios do conjunto fixo de repositórios em cada era e outra com as diferenças exatas entre eras. Todos os artefatos intermediários, SHAs, hashes, decisões

Tabela 1. Questões de pesquisa e medidas associadas no paradigma GQM.

Q	Questão	Medidas
Q1	Como a estrutura interna dos mesmos projetos válidos varia entre eras?	Médias das sete métricas CK por era e diferenças exatas por grupo ($n = 17$ e $n = 453$).
Q2	Quais mudanças acompanham transições declaradas de Spring e Boot?	Dependências Maven/Gradle em cada SHA; variações longitudinais de CBO.
Q3	Quais diferenças estruturais aparecem entre projetos válidos com e sem evidências detectadas de IA em E4?	Vestígios em arquivos e commits até o snapshot ($N = 470$); médias CK e porte.
Q4	Como a curadoria sem filtro de estrelas, as faixas de popularidade e o porte contextualizam as métricas?	Curadoria nos 4 cenários, distribuição de estrelas, arquivos e LOC.

por cenário do RepoReaper e resultados brutos de extração permanecem preservados para auditoria independente.

5. Resultados e Discussão

5.1. Amostra final válida e relevância sem filtro de estrelas

A filtragem de metadados com atividade na era E4 (`pushed_at >= 2022-01-01T00:00:00Z`) e corte temporal de criação até 2017 selecionou 785 repositórios candidatos (22 criados até 2013 e 763 criados entre 2014 e 2017), dos quais 607 foram aprovados localmente na curadoria conjunta dos quatro cenários do RepoReaper (C1 + C2 Ajustado + C3 + C4). Aplicada a exigência metodológica de extração válida das métricas CK em todas as eras previstas para cada grupo temporal, consolidou-se a amostra final de **470 repositórios válidos** e **1.427 snapshots analisados** (Tabela 2): **17 repositórios** no grupo pioneiro (2009–2013), medidos continuamente nas quatro eras (E_1, E_2, E_3, E_4), e **453 repositórios** no grupo principal (2014–2017), medidos continuamente nas três eras (E_2, E_3, E_4).

Tabela 2. Síntese da filtragem e composição da amostra longitudinal válida com extração CK completa em todas as eras de cada grupo ($N = 470$ repositórios).

Grupo	Eras Exigidas	Candidatos	RepoReaper	Válidos (CK)	Snapshots
Pioneiro (2009–2013)	E1, E2, E3 e E4	22	20	17	68
Principal (2014–2017)	E2, E3 e E4	763	587	453	1.359
Total		785	607	470	1.427

A adoção dos quatro cenários do RepoReaper em substituição ao filtro arbitrário por número de estrelas preservou elevada representatividade do ecossistema na amostra final: entre os 470 sistemas válidos, **209 (44,5%) possuem menos de dez estrelas** e **261 (55,5%) possuem dez ou mais estrelas**. Em E4, o confronto estrutural entre os repositórios válidos com menos de dez estrelas ($n = 209$) e aqueles com dez ou mais estrelas ($n = 261$) confirma empiricamente a adequação dessa escolha metodológica: ambos os estratos exibem métricas intrínsecas em nível de classe e método muito próximas — CBO médio de 5,483 contra 5,830, tamanho por método de 5,568 contra 5,707 linhas, profundidade de herança (DIT) de 1,200 contra 1,254 e falta de coesão (LCOM*) de 0,232 contra

0,239 —, diferindo substancialmente apenas na escala física total dos projetos (média de 88,0 arquivos Java e 3.821 LOC nos projetos com menos de dez estrelas, frente a 543,1 arquivos e 41.054 LOC nos mais populares). Isso comprova que repositórios de menor visibilidade social aprovados pelo RepoReaper e ativos ao longo das eras constituem sistemas Java consistentes e bem estruturados, e não artefatos triviais.

5.2. Evolução longitudinal das métricas CK por grupo temporal

Como todos os repositórios válidos possuem extração CK completa em todas as eras do seu respectivo grupo, as tabelas de valores médios por era e as tabelas de diferenças entre eras incidem exatamente sobre o mesmo conjunto invariável de sistemas, garantindo identidade perfeita entre a variação média dos projetos e a diferença direta entre as médias de cada período.

Para o **grupo pioneiro** ($n = 17$ repositórios válidos criados até 2013 e acompanhados nas quatro eras), a Tabela 3 apresenta os valores médios das sete métricas de E1 a E4, enquanto a Tabela 4 consolida as diferenças de E1 frente a E2, E3 e E4 sobre os mesmos 17 projetos. Observa-se crescimento monotônico e expressivo no acoplamento médio entre classes (CBO: de 4,878 em E1 para 5,394 em E2, 5,626 em E3 e 5,803 em E4, acumulando $+0,516$, $+0,748$ e $+0,925$) e no conjunto de resposta por classe (RFC: de 7,639 em E1 para 8,387 em E2, 8,972 em E3 e 9,956 em E4, acumulando $+0,748$, $+1,333$ e $+2,317$). Em sentido oposto, a complexidade ponderada por classe (WMC) recuou continuamente ao longo das quatro eras (de 8,094 em E1 para 7,201 em E4, variação acumulada de $-0,893$), e a falta de coesão (LCOM*) diminuiu de 0,298 para 0,249 ($-0,049$), indicando classes internamente mais coesas apesar da expansão física média desses sistemas (de 1.126,5 arquivos Java e 63.102 LOC em E1 para 2.999,6 arquivos e 228.571 LOC em E4). Os indicadores hierárquicos (DIT e NOC) mantiveram-se estáveis em todo o intervalo.

Tabela 3. Grupo pioneiro (criados até 2013, $n = 17$): médias das métricas CK dos mesmos 17 repositórios válidos em E1, E2, E3 e E4.

Métrica	E1	E2	E3	E4
CBO	4,878	5,394	5,626	5,803
RFC	7,639	8,387	8,972	9,956
WMC	8,094	8,076	7,936	7,201
LOC/método	6,316	5,957	6,208	6,446
DIT	1,416	1,355	1,376	1,341
NOC	0,083	0,068	0,104	0,081
LCOM*	0,298	0,276	0,277	0,249

Tabela 4. Grupo pioneiro (criados até 2013, $n = 17$): diferenças das médias de E1 em relação às eras subsequentes nos mesmos 17 repositórios válidos.

Métrica	(E2 − E1)	(E3 − E1)	(E4 − E1)
CBO	+0,516	+0,748	+0,925
RFC	+0,748	+1,333	+2,317
WMC	-0,019	-0,158	-0,893
LOC/método	-0,359	-0,108	+0,130
DIT	-0,062	-0,040	-0,076
NOC	-0,015	+0,021	-0,002
LCOM*	-0,022	-0,021	-0,049

Tabela 5. Grupo principal (criados entre 2014 e 2017, $n = 453$): médias das métricas CK dos mesmos 453 repositórios válidos em E2, E3 e E4.

Métrica	E2	E3	E4
CBO	5,405	5,591	5,671
RFC	6,603	7,224	7,587
WMC	5,819	5,801	5,759
LOC/método	5,308	5,489	5,616
DIT	1,221	1,229	1,226
NOC	0,059	0,065	0,063
LCOM*	0,245	0,243	0,235

Tabela 6. Grupo principal (criados entre 2014 e 2017, $n = 453$): diferenças das médias entre E2, E3 e E4 nos mesmos 453 repositórios válidos.

Métrica	(E3 − E2)	(E4 − E3)	(E4 − E2)
CBO	+0,186	+0,080	+0,266
RFC	+0,622	+0,363	+0,984
WMC	-0,018	-0,042	-0,060
LOC/método	+0,181	+0,127	+0,307
DIT	+0,008	-0,003	+0,005
NOC	+0,006	-0,002	+0,004
LCOM*	-0,002	-0,007	-0,009

Para o **grupo principal** ($n = 453$ repositórios válidos criados entre 2014 e 2017 e acompanhados continuamente nas três eras), a Tabela 5 exibe os valores médios em E2, E3 e E4, e a Tabela 6 apresenta as diferenças entre eras adjacentes (E2 vs. E3 e E3 vs. E4) e no horizonte completo (E2 vs. E4), todas calculadas rigorosamente sobre os mesmos 453 repositórios:

- **E2 vs. E3:** O acoplamento médio (CBO) elevou-se em $+0,186$ (de 5,405 para 5,591) e o RFC cresceu $+0,622$ (de 6,603 para 7,224). Em contrapartida, o WMC registrou leve retração de $-0,018$ (de 5,819 para 5,801) e o LCOM* recuou $-0,002$ (de 0,245 para 0,243), com aumento moderado de $+0,181$ linhas no tamanho por método e estabilidade em DIT ($+0,008$) e NOC ($+0,006$).

- **E3 vs. E4:** O CBO avançou mais $+0,080$ (de 5,591 para 5,671) e o RFC aumentou $+0,363$ (de 7,224 para 7,587). O WMC voltou a recuar ($-0,042$, atingindo 5,759) e o LCOM* reduziu mais $-0,007$ (chegando a 0,235), ao passo que o LOC por método cresceu $+0,127$ e DIT ($-0,003$) e NOC ($-0,002$) permaneceram praticamente constantes.
- **E2 vs. E4:** Na trajetória acumulada entre as três eras, os mesmos 453 repositórios apresentaram elevação consistente de $+0,266$ em CBO e de $+0,984$ em RFC, coexistindo com redução acumulada de $-0,060$ em WMC e de $-0,009$ em LCOM*, acréscimo de $+0,307$ linhas por método e conservação da estrutura de herança (DIT $+0,005$; NOC $+0,004$). No mesmo intervalo, o porte contextual médio expandiu-se de 121,7 arquivos Java e 7.492 LOC em E2 para 185,7 arquivos e 11.703 LOC em E3 e 241,0 arquivos e 16.839 LOC em E4.

A convergência de sinal entre o grupo pioneiro ($n = 17$) e o grupo principal ($n = 453$) demonstra que, à medida que sistemas Spring amadurecem e expandem sua base de código, a colaboração e o acoplamento entre classes (CBO e RFC) aumentam de maneira gradual, sem degradar a coesão interna dos atributos (LCOM*) nem inflacionar a complexidade média das classes (WMC).

5.3. Presença de Spring Boot e evidências de assistência por IA

Nos repositórios válidos com inspeção estática de manifestos de dependência entre eras adjacentes, a passagem de ausência para presença declarada de dependências `org.springframework.boot` foi observada em sete transições E1–E2 (variação média de CBO de $+0,585$), 32 transições E2–E3 ($+0,541$) e 11 transições E3–E4 ($-0,020$). A heterogeneidade de magnitude e sinal entre os períodos, somada ao fato de declarações de Boot frequentemente integrarem módulos específicos em projetos multimódulo, indica que a adoção declarada do framework não produz, isoladamente, um deslocamento estrutural uniforme sobre o código.

Quanto aos vestígios de assistência por inteligência artificial na amostra final de **470 repositórios válidos** em E4, **75 projetos (16,0%) apresentaram evidência explícita detectada** em arquivos de configuração ou histórico de commits e **395 projetos (84,0%) não apresentaram os sinais inspecionados** (Tabela 7). Nenhum vestígio explícito dos agentes monitorados foi identificado nos snapshots históricos de E1 a E3, o que não comprova inexistência de ferramentas assistivas preliminares.

Tabela 7. E4: médias entre os repositórios válidos finais ($N = 470$) com evidência detectada de IA (A, $n_a = 75$) e sem evidência detectada (B, $n_b = 395$).

Métrica	Média A	Média B	A menos B
CBO	6,101	5,595	+0,506
RFC	9,840	7,261	+2,579
WMC	6,960	5,593	+1,366
LOC/método	6,182	5,544	+0,639
DIT	1,301	1,216	+0,085
NOC	0,079	0,061	+0,018
LCOM*	0,251	0,233	+0,018

Em E4, o subgrupo com evidência detectada de IA ($n_a = 75$) registrou médias

aritméticas superiores nas sete medidas estruturais frente ao subgrupo sem evidência detectada ($n_b = 395$) — com destaque para CBO ($6,101$ contra $5,595$; $+0,506$), RFC ($9,840$ contra $7,261$; $+2,579$), WMC ($6,960$ contra $5,593$; $+1,366$) e tamanho por método ($6,182$ contra $5,544$; $+0,639$). Contudo, a inspeção contextual de porte evidencia forte assimetria de escala entre os dois subconjuntos: os 75 projetos válidos com evidência de IA concentram, em média, **113.864 LOC e 1.341,6 arquivos Java**, contra **7.529 LOC e 150,7 arquivos Java** nos 395 projetos sem evidência detectada. Sem controle multivariado por porte, domínio e tamanho de equipe, essas diferenças descrevem o perfil dos sistemas que incorporaram artefatos explícitos de agentes, não autorizando inferir que a IA eleve ou degrade causalmente a complexidade estrutural.

O acompanhamento longitudinal dos mesmos 470 projetos válidos entre E3 e E4 corrobora essa interpretação: nos 75 sistemas que exibiram vestígios de IA em E4, o CBO médio passou de 5,858 em E3 para 6,101 em E4 ($+0,243$), enquanto nos 395 sistemas sem evidência detectada evoluiu de 5,542 em E3 para 5,595 em E4 ($+0,053$). Verifica-se que o subgrupo com evidência de IA em E4 já operava em patamar superior de acoplamento na era anterior (E3), indicando que projetos de maior escala e atividade recente tendem simultaneamente a exibir maior interconexão entre classes e maior propensão à adoção visível de agentes de codificação.

5.4. Sumarização dos resultados frente aos objetivos

Objetivo específico (i): trajetórias estruturais. Tanto nos 17 projetos válidos do grupo pioneiro acompanhados de E1 a E4 quanto nos 453 projetos válidos do grupo principal acompanhados de E2 a E4, verificou-se expansão monotônica do acoplamento entre classes (CBO: $+0,925$ e $+0,266$) e da superfície de resposta (RFC: $+2,317$ e $+0,984$), coexistindo com retração da complexidade ponderada por classe (WMC: $-0,893$ e $-0,060$), redução da falta de coesão (LCOM*: $-0,049$ e $-0,009$) e estabilidade das hierarquias de herança (DIT e NOC).

Objetivo específico (ii): configurações de Spring e Boot. As transições observáveis para presença declarada de Spring Boot em manifestos Maven/Gradle exibiram variações distintas entre eras adjacentes ($+0,585$, $+0,541$ e $-0,020$ em CBO), mostrando que a introdução do framework interage com a arquitetura modular de cada projeto sem impor direção única às métricas CK.

Objetivo específico (iii): evidências de IA. Na amostra final de 470 projetos válidos em E4, os 75 sistemas com evidência detectada de IA apresentaram médias superiores nas sete métricas em comparação aos 395 sistemas sem evidência detectada, mas reuniram bases de código quinze vezes maiores em LOC médio (113.864 contra 7.529 LOC) e já partiam de acoplamento mais elevado em E3 (5,858 contra 5,542), caracterizando associação com escala e ritmo de evolução recente em vez de efeito causal isolado.

Objetivo específico (iv): contexto de curadoria, estrelas e porte. A curadoria pelos quatro cenários do RepoReaper (com flexibilização de C2 para convenções do Spring Boot) aliada à exigência de completude longitudinal consolidou 470 repositórios válidos, demonstrando que projetos com menos de dez estrelas (44,5% da amostra final) mantêm coesão, acoplamento e tamanho de métodos equivalentes aos projetos populares.

Objetivo geral. Caracterizou-se longitudinalmente a evolução estrutural de 1.427 snapshots completos distribuídos em 470 projetos válidos do ecossistema Spring ao longo

de quatro eras tecnológicas, evidenciando que o crescimento contínuo dos sistemas eleva a comunicação entre classes (CBO e RFC), mas preserva o controle de complexidade interna (WMC), a coesão de atributos (LCOM*) e hierarquias enxutas de herança.

5.5. Ameaças à validade

Construto. A análise estática do CK opera sobre o código-fonte sem resolução de dependências externas compiladas, o que pode subestimar tipos herdados ou acoplamentos indiretos. LCOM* é indefinido para classes sem atributos ou métodos compatíveis, sendo omitido apenas do denominador correspondente sem imputação por zero. A detecção de IA baseia-se em rastros explícitos versionados (arquivos de configuração, identidades e coautoria); o uso silencioso de completamento inline ou chats externos não deixa artefato no Git, de modo que o grupo sem evidência detectada não deve ser interpretado como exclusivamente humano. Por fim, a curadoria pelos quatro cenários do RepoReaper inspeciona metadados e estrutura de build/testes consolidados, e a flexibilização de `demo`, `sample` e `example` no Cenário C2 constitui adaptação operacional para o domínio Spring Boot, sem substituir inspeção qualitativa individual.

Interna. Variações entre eras e diferenças entre subgrupos com e sem evidência de IA coexistem com expansão acentuada de porte (LOC e número de arquivos), refatorações arquiteturais, rotatividade de desenvolvedores e diferenças na recência do último commit dentro de E4. A ausência de piso mínimo de porte permite que fases iniciais enxutas e sistemas corporativos extensos coexistam nas médias descritivas, razão pela qual os indicadores CK devem ser lidos em conjunto com as medidas contextuais de tamanho e com as diferenças calculadas sobre os mesmos repositórios.

Externa. A descoberta inicial depende do tópico `spring-boot` declarado atualmente no GitHub, de linguagem principal Java, da exclusão de repositórios arquivados, da exigência de atividade a partir de 2022 (`pushed_at`) e da presença de snapshots com extração CK válida em todas as eras do grupo temporal, introduzindo viés de sobrevivência e continuidade em favor de projetos longevos e regularmente mantidos. Além disso, o grupo pioneiro anterior a 2014 é reduzido (17 projetos válidos), impedindo generalização irrestrita para todo o universo histórico de aplicações Spring tradicionais ou privadas.

Conclusão. O estudo adota estatística estritamente descritiva, sem testes de hipótese, p-valores ou tamanhos de efeito não paramétricos. Como os mesmos repositórios são observados repetidamente entre todas as eras de seu grupo, as medições longitudinais não constituem amostras independentes; a consistência das trajetórias apoia-se na eliminação completa de atrito amostral entre períodos dentro de cada grupo ($n = 17$ em E1–E4 e $n = 453$ em E2–E4).

6. Conclusão

Este trabalho caracterizou longitudinalmente a evolução da qualidade estrutural de projetos Java do ecossistema Spring criados até 2017 e ativos na era contemporânea (E4, `pushed_at >= 2022-01-01`), acompanhando versões históricas com rastreabilidade estrita por commit Git até outubro de 2026. Em substituição a filtros arbitrários de popularidade por número de estrelas, a relevância de engenharia dos 785 candidatos iniciais foi aferida localmente pela aplicação simultânea dos quatro cenários do RepoReaper (C1 de finalidade não didática, C2 ajustado para projetos brinquedo com flexibilização

dos termos `demo`, `sample` e `example` próprios de convenções do Spring Boot, C3 de automação de build Maven/Gradle e C4 de presença de testes automatizados), aprovando 607 repositórios. A exigência metodológica de dados CK válidos em todas as eras previstas para cada repositório consolidou a amostra final de **470 repositórios válidos (1.427 snapshots CK)**, organizados em 17 projetos do grupo pioneiro (2009–2013, acompanhados continuamente nas quatro eras E1–E4) e 453 projetos do grupo principal (2014–2017, acompanhados continuamente nas três eras E2–E4).

Objetivo (i), trajetórias estruturais. O acoplamento entre objetos (CBO) e o conjunto de resposta das classes (RFC) cresceram de forma monotônica nos mesmos projetos ao longo das eras: $+0,925$ em CBO e $+2,317$ em RFC nos 17 repositórios pioneiros entre E1 e E4, e $+0,266$ em CBO e $+0,984$ em RFC nos 453 repositórios do grupo principal entre E2 e E4. Em contrapartida, a complexidade ponderada por classe (WMC) apresentou retração ($-0,893$ no grupo pioneiro e $-0,060$ no grupo principal) e a falta de coesão (LCOM*) diminuiu ($-0,049$ e $-0,009$), acompanhadas de alta estabilidade na hierarquia de herança (DIT e NOC). Os achados demonstram que a expansão funcional dos sistemas eleva a interdependência entre componentes sem provocar degradação na coesão interna nem inflação na complexidade média das classes.

Objetivo (ii), Spring e Boot. As transições declaradas para dependências do Spring Boot entre eras adjacentes não exibiram direção estrutural uniforme nas métricas CK, evidenciando que a adoção do framework opera em interação com o desenho modular de cada sistema, sem atuar isoladamente como fator determinístico de simplificação ou complexidade.

Objetivo (iii), evidências de IA. Em E4, entre os 470 repositórios válidos finais, os 75 sistemas com evidência detectada de assistência por IA apresentaram médias superiores nas sete medidas estruturais em relação aos 395 sistemas sem evidência detectada. Contudo, esses projetos possuíam porte médio substancialmente superior (113.864 contra 7.529 LOC) e já registravam acoplamento mais elevado na era anterior E3 (CBO de 5,858 contra 5,542 nos mesmos grupos), indicando que a presença de artefatos de agentes associa-se à escala e à intensidade de evolução recente dos sistemas, sem caracterizar efeito causal direto sobre a arquitetura.

Objetivo (iv), curadoria, estrelas e porte. A exigência de presença contínua de métricas CK eliminou variações de composição entre eras tanto no grupo pioneiro ($n = 17$ em todas as comparações de E1–E4) quanto no grupo principal ($n = 453$ em E2 vs. E3, E3 vs. E4 e E2 vs. E4). Demonstrou-se empiricamente que projetos com menos de dez estrelas validados pelo protocolo (44,5% da amostra final) apresentam coesão, acoplamento e tamanho de métodos equivalentes aos de repositórios altamente estrelados, confirmando a adequação de critérios de engenharia em lugar de métricas de popularidade social.

Como contribuição central, o estudo entrega um protocolo longitudinal auditável e reprodutível para mineração do ecossistema Spring Boot, aliando curadoria local em quatro dimensões adaptada às convenções do framework, completude longitudinal de snapshots por era e análise estrutural sobre conjuntos fixos de projetos. Trabalhos futuros podem investigar o papel específico de estereótipos arquiteturais do Spring sobre o acoplamento e aprofundar a rastreabilidade de código assistido por IA em granularidade de

método e pull request, observados os limites descritos na Seção 5.5.

Referências

Aniche, M. (2015). Java Code Metrics Calculator (CK). `https://github.com/mauricioaniche/ck`. Revisão de desenvolvimento 0.7.1-SNAPSHOT, commit 8a1ef916d597d33ad619d748b616bb7dbc3f0c36; adaptador local com fórmulas preservadas.

Aniche, M., Treude, C., Zaidman, A., van Deursen, A., and Gerosa, M. A. (2016). SATT: Tailoring code metric thresholds for different software architectures. In *2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 41–50. IEEE.

Basili, V. R., Caldiera, G., and Rombach, H. D. (1994). The goal question metric approach. In *Encyclopedia of Software Engineering*, pages 528–532. John Wiley & Sons, New York.

Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

Child, M., Rosner, P., and Counsell, S. (2019). A comparison and evaluation of variants in the coupling between objects metric. *Journal of Systems and Software*, 151:120–132.

Dey, T., Mousavi, S., Ponce, E., Godfrey, T., Padhy, B., Nguyen, A., and Mockus, A. (2020). Detecting and characterizing bots that commit code. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR 2020)*, pages 209–219, Seoul, Republic of Korea. ACM.

GitHub (2021). Introducing github copilot: your ai pair programmer. `https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/`. Publicado em 29 de junho de 2021.

Godfrey, M. W. and Tu, Q. (2000). Evolution in open source software: A case study. In *Proceedings of the International Conference on Software Maintenance*. IEEE.

Hassan, A. E. (2008). The road ahead for Mining Software Repositories. In *2008 Frontiers of Software Maintenance*, pages 48–57. IEEE.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., and Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79.

ISO/IEC (2011). Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models. International Standard ISO/IEC 25010:2011, International Organization for Standardization, Geneva.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2014). The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR '14)*, pages 92–101, Hyderabad, India. ACM.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2016). An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates.

Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., and Jiang, Z. M. (2023). GitHub Copilot AI pair programmer: asset or liability? *Journal of Systems and Software*, 203:111734.

Mozannar, H., Bansal, G., Fourney, A., and Horvitz, E. (2024). Reading between the lines: Modeling user behavior and costs in AI-assisted programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–18. ACM.

Munaiah, N., Kroh, S., Cabrey, C., and Nagappan, M. (2017). Curating GitHub for engineered software projects. *Empirical Software Engineering*, 22(6).

Oliveira, J., Viggiato, M., Pinheiro, D., and Figueiredo, E. (2021). Mining experts from source code analysis: An empirical evaluation. *Journal of Software Engineering Research and Development*, 9.

Park, R. E. (1992). Software size measurement: A framework for counting source statements. Technical Report CMU/SEI-92-TR-020, Software Engineering Institute, Carnegie Mellon University.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. (2023). The impact of AI on developer productivity: evidence from GitHub Copilot. arXiv:2302.06590.

Pressman, R. S. and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, 9 edition.

Radjenović, D., Heričko, M., Torkar, R., and Živković, A. (2013). Software fault prediction metrics: a systematic literature review. *Information and Software Technology*, 55(8):1397–1418.

Robbes, R., Matricon, T., Degueule, T., Hora, A., and Zacchiroli, S. (2026). Agentic much? adoption of coding agents on GitHub. arXiv:2601.18341v2. Versão de 8 de abril de 2026. Consultado em 11 de outubro de 2026.

Singh, Y., Kaur, A., and Malhotra, R. (2010). Empirical validation of object-oriented metrics for predicting fault proneness models. *Software Quality Journal*, 18(1):3–35.

Spring (2014). Spring boot 1.0 ga released. `https://spring.io/blog/2014/04/01/spring-boot-1-0-ga-released/`. Publicado em 1 de abril de 2014.

Sun, P., Kim, D.-K., Hua, M., and Lu, L. (2022). Measuring impact of dependency injection on software maintainability. *Computers*, 11(9):141.

Yetistiren, B., Ozsoy, I., Ayerdem, M., and Tuzun, E. (2023). Evaluating the code quality of AI-assisted code generation tools: an empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778.

Apêndice A — Dados, curadoria local e reprodução do painel

Os artefatos empíricos desta investigação derivam da base de descoberta `Instrumentos/Codigos/Artefatos/selecao_spring_boot/runs/20260920_java/population.csv` (95.894 registros únicos), da execução longitudinal auditada `Instrumentos/Codigos/Artefatos/evolucao_spring/runs/20261015_base_original_ate2017/` e da consolidação expandida da janela E4 (`pushed_at >= 2022-01-01T00:00:00Z`), que reúne os 785 candidatos criados até 2017 sem filtro por contagem de estrelas. Não há sorteio nem substituição de identificadores em nenhuma etapa do protocolo.

A higienização local pelos quatro cenários conjuntos do RepoReaper (C1 de finalidade não didática, C2 ajustado sem penalizar `demo`/`sample`/`example`, C3 de arquitetura de build Maven/Gradle e C4 de presença de testes automatizados) é executada pelo instrumento `reporeaper_curation.py`, registrando as decisões por cenário, as *flags* lexicais acionadas e os indicadores estruturais de build e testes para cada um dos 785 candidatos (607 aprovados na curadoria). A filtragem subsequente de completude longitudinal retém exclusivamente os repositórios com extração válida de métricas CK em todas as eras previstas para o seu respectivo grupo temporal, definindo os **470 repositórios válidos finais**: 17 no grupo pioneiro (medidos em E1, E2, E3 e E4; 68 snapshots) e 453 no grupo principal (medidos em E2, E3 e E4; 1.359 snapshots), perfazendo 1.427 snapshots analisados.

Os arquivos de medição registram cada posição projeto–era com identificador do repositório, nome completo, contagem de estrelas, grupo temporal, era, SHA do commit avaliado, status de recuperação e extração, quantidade de arquivos Java, linhas físicas de código (contadas uma vez por arquivo como contexto descritivo, sem duplicação de classes internas), sete métricas estruturais do CK (com separação entre produção e testes nos dados brutos) e classificação temporalizada de vestígios de IA em E4. Falhas de clonagem, ausência de commit no intervalo da era ou ausência de arquivos Java permanecem registradas nos artefatos de triagem e nunca são convertidas em métrica zero, assim como valores indefinidos de LCOM* são retirados apenas do denominador correspondente.

As tabelas de valores e de diferenças operam estritamente sobre os mesmos identificadores válidos em cada grupo ($n = 17$ em todas as comparações de E1 a E4; $n = 453$ em E2 vs. E3, E3 vs. E4 e E2 vs. E4; e $N = 470$ na comparação de vestígios de IA em E4, com 75 projetos com evidência detectada e 395 sem evidência detectada). Manifestos JSON e arquivos CSV intermediários conservam os hashes dos instrumentos, parâmetros de execução e contagens reconciliáveis para verificação e reanálise independente.